LAYER 2: STATE & EXECUTION LAYER — VERTICAL DECOMPOSITION FOR AGENTIC CODING

Purpose: Immutable smart contracts handling core quadratic voting logic,

voice credit ledger management, and proposal execution.

Architecture Pattern: Modular, deterministic, non-upgradable core with

clear agent boundaries and single-responsibility modules.

VERTICAL 1: VOICE CREDIT MANAGEMENT MODULE (VCM)

Agent: CreditLedgerAgent

Responsibility: Account for every voice credit — mint, burn, transfer, lock

Sub-Module 1.1: Credit Registry

- **State Variables:**
  - `mapping(address => uint256) voiceCreditBalance`
  - `mapping(address => uint256) lockedCredits` (credits committed to active votes)
  - `mapping(address => uint256) lastRegenerationTimestamp`
  - `uint256 globalCreditPool`
  - `uint256 regenerationRate` (credits per block/epoch)
  - `uint256 maxCreditCap` (per-identity ceiling)
- **Core Functions:**
  - `allocateInitialCredits(address identity, uint256 amount)` → One-time allocation upon verified identity registration
  - `regenerateCredits(address identity)` → Time-based regeneration: adds min(regenerationRate, maxCreditCap - current)
  - `lockCredits(address identity, uint256 amount)` → Atomically deduct from balance, add to lockedCredits

  - `unlockCredits(address identity, uint256 amount)` → Return locked credits to balance (post-vote or proposal cancellation)
  - `slashCredits(address identity, uint256 amount, bytes32 reasonHash)` → Penalty for Sybil detection or rule violation
- **Invariants (MUST NEVER VIOLATE):**
  - a: voiceCreditBalance[a] + lockedCredits[a] maxCreditCap
  - a: voiceCreditBalance[a] 0 (checked arithmetic)
  - globalCreditPool = Σ(voiceCreditBalance[i] + lockedCredits[i])
- **Agent Prompt for Implementation:** "Implement a VoiceCreditRegistry contract with the above state and functions. Use OpenZeppelin's ReentrancyGuard. All state transitions must emit events. Include comprehensive unit tests for all invariant violations."

Sub-Module 1.2: Credit Math Engine

- **Pure Functions (no state access):**
  - `calculateQuadraticCost(int256[] voteVector) → uint256 totalCost` → Formula: Σ(v ²) for each option i → Must handle negative votes (opposition) as: v ² (sign preserved separately)
  - `calculateVotePower(uint256 creditsSpent, uint256 optionIndex) → int256 votePower` → Inverse: √credits allocated to option = vote power
  - `verifyBudgetConstraint(uint256[] allocation, uint256 availableCredits) → bool` → Ensures Σ(allocation) availableCredits
- **Agent Prompt:** "Implement VoiceCreditMath as a pure library. Use fixed-point arithmetic with 18 decimal precision. Include overflow/underflow guards."

VERTICAL 2: PROPOSAL LIFECYCLE MODULE (PLM)

Agent: ProposalLifecycleAgent

Responsibility: CRUD for proposals, state transitions, validation

Sub-Module 2.1: Proposal Factory & Registry

- **State Variables:**
  - `mapping(bytes32 => Proposal) proposals`
  - `bytes32[] activeProposals`
  - `bytes32[] executedProposals`
  - `bytes32[] failedProposals`
  - `uint256 proposalCounter`
- **Proposal Struct:**

```
struct Proposal {
    bytes32 id;
    address proposer;
    string title;
    string descriptionURI;        // IPFS hash
    bytes executionPayload;       // ABI-encoded call data
    address targetContract;       // Contract to call if passed
```

```
    uint256 creationBlock;
    uint256 votingStartBlock;
    uint256 votingEndBlock;
    uint256 executionDelayBlocks; // Layer 5 timelock integration
    uint256 quorumThreshold;      // Dynamic, from Layer 3
    ProposalStatus status;
    mapping(uint256 => int256) voteTallies;  // optionIndex → net vote power
    mapping(address => VoteReceipt) votes;
}

struct VoteReceipt {
    uint256[] creditAllocations;  // per-option credits spent
    int256[] votePowers;          // per-option derived vote power
    uint256 totalCreditsSpent;
    uint256 blockNumber;
}

enum ProposalStatus {
    Draft,
    Active,      // Open for voting
    Passed,      // Quorum met, majority achieved
    Failed,      // Quorum not met OR majority against
    Timelocked,  // Passed, awaiting Layer 5 veto period
    Executed,    // Successfully executed
    Cancelled,   // Vetoed or invalidated
    Expired      // Timelock expired without execution
}
```

- **Core Functions:**
  - `createProposal(string title, string descURI, bytes payload, address target)` → Returns proposalId, emits ProposalCreated → Validates: proposer has minimum credits, payload is non-empty
  - `activateProposal(bytes32 proposalId)` → Transitions Draft → Active at votingStartBlock
  - `finalizeProposal(bytes32 proposalId)` → Calculates results, transitions to Passed/Failed → Checks: block.number > votingEndBlock, status == Active
  - `executeProposal(bytes32 proposalId)` → Transitions Passed → Timelocked → Executed → Delegates actual execution to Execution Engine (Vertical 3)
- **Agent Prompt:** "Implement ProposalRegistry with the full Proposal struct and lifecycle. Ensure atomic state transitions. Add comprehensive access control. Include batch query functions for frontend integration."

Sub-Module 2.2: Proposal Validator

- **Pure Validation Functions:**
  - `validatePayload(bytes payload, address target) → (bool valid, string error)` → Simulates call to target with payload (staticcall) → Checks target is not Layer 2 core contract (self-protection)
  - `validateQuorum(bytes32 proposalId) → bool` → $\Sigma|\text{votePower}| \quad \text{quorumThreshold}$
  - `validateMajority(bytes32 proposalId) → bool` → net positive votes > net negative votes (for

binary) → Or plurality wins (for multi-option)

- **Agent Prompt:** "Implement ProposalValidator as a pure library with static analysis helpers. Include payload safety checks against known attack vectors."

VERTICAL 3: VOTE EXECUTION ENGINE (VEE)

Agent: VoteExecutionAgent

Responsibility: Process votes, apply quadratic math, update tallies atomically

Sub-Module 3.1: Vote Processor

- **Core Function:**
  - `castVote(bytes32 proposalId, uint256[] creditAllocations, int256[] voteSigns)`

    Step-by-Step Atomic Flow:

    1. VERIFY: msg.sender has verified identity (Layer 1 integration)
    2. VERIFY: proposal.status == Active && block.number within window
    3. VERIFY: creditAllocations.length == proposal.options.length
    4. COMPUTE: totalCost = VoiceCreditMath.calculateQuadraticCost(creditAllocations)
    5. VERIFY: totalCost voiceCreditBalance[msg.sender] (unlocked)
    6. LOCK: VoiceCreditRegistry.lockCredits(msg.sender, totalCost)
    7. COMPUTE: votePowers[i] = voteSigns[i] * √creditAllocations[i]
    8. UPDATE: proposal.voteTallies[i] += votePowers[i]
    9. STORE: proposal.votes[msg.sender] = VoteReceipt(…)
    10. EMIT: VoteCast(proposalId, msg.sender, totalCost, votePowers)
- **Edge Cases:**
  - Vote change: Allow voter to update allocation (refund old, charge new)
  - Partial abstention: creditAllocations[i] = 0 for abstained options
  - Negative voting: voteSigns[i] = -1 for opposition
- **Agent Prompt:** "Implement VoteProcessor with the exact atomic flow above. Use checks-effects-interactions pattern. Prevent reentrancy. Include vote change functionality with proper credit reconciliation."

Sub-Module 3.2: Tally Engine

- **Functions:**
  - `computeFinalTally(bytes32 proposalId) → TallyResult`

```
struct TallyResult {
    uint256 totalParticipants;
    uint256 totalCreditsSpent;
    int256[] netVotePowers;
    uint256 winningOptionIndex;
    bool quorumMet;
    bool majorityMet;
}
```

  - `computeQuadraticMedian(bytes32 proposalId) → int256` → For continuous parameter adjustments (Layer 3 optimization input)

- **Agent Prompt:** "Implement TallyEngine with gas-efficient tally computation. Support both binary and multi-option proposals. Include merkle-root generation for off-chain verification."

VERTICAL 4: TREASURY EXECUTION MODULE (TEM)

Agent: TreasuryExecutionAgent

Responsibility: Safely execute passed proposals, manage treasury interactions

Sub-Module 4.1: Execution Dispatcher

- **State:**
  - `mapping(bytes32 => bool) executedProposals`
  - `address treasuryContract` (external, set at deployment)
- **Core Function:**
  - `dispatchExecution(bytes32 proposalId)` → Only callable after timelock expires and no veto active → Verifies: proposal.status == Passed, timelock expired, !executedProposals[proposalId] → Performs: low-level call to proposal.targetContract with proposal.executionPayload → On success: mark executed, emit ProposalExecuted → On failure: emit ExecutionFailed, allow retry with adjusted gas
- **Safety Constraints:**
  - NEVER call back into Layer 2 core contracts
  - ALWAYS verify call target is not Layer 2 address
  - Use delegatecall ONLY for pre-approved module contracts
- **Agent Prompt:** "Implement ExecutionDispatcher with strict safety checks. Include execution retry mechanism with exponential backoff. Add comprehensive event logging for audit trails."

Sub-Module 4.2: Treasury Guard

- **Functions:**
  - `validateTreasuryImpact(bytes payload, address target) → (bool safe, uint256 maxSpend)` → Static analysis of payload to extract value transfers → Compare against treasury balance and daily spend limits
  - `enforceSpendLimit(uint256 amount) → bool` → Layer 3 parameter: dailySpendLimit
- **Agent Prompt:** "Implement TreasuryGuard with payload analysis helpers. Integrate with Layer 3 parameter bounds for dynamic limits."

VERTICAL 5: PARAMETER INTERFACE MODULE (PIM)

Agent: ParameterInterfaceAgent

Responsibility: Receive Layer 3 optimizations, validate bounds, apply changes

Sub-Module 5.1: Parameter Registry

- **State:**

```
struct SystemParameters {
    uint256 quorumBase;           // Base quorum percentage (e.g., 400 = 4%)
    uint256 quorumDynamicFactor;  // Layer 3 adjustment multiplier
    uint256 votingDurationBlocks; // Default voting window
    uint256 creditRegenerationRate; // Credits per epoch
    uint256 maxCreditCap;         // Per-identity ceiling
    uint256 proposalStake;        // Credits required to create proposal
    uint256 timelockDuration;     // Layer 5 veto window
    uint256 dailySpendLimit;      // Treasury guard limit
}
```

- **Core Functions:**
  - `getEffectiveQuorum()` → `uint256` → Returns: quorumBase * quorumDynamicFactor / 10000
  - `applyParameterUpdate(bytes32 paramKey, uint256 newValue, bytes32 layer3Proof)` → Verifies: call originates from Layer 5 timelock (not direct Layer 3) → Verifies: newValue within hardcoded bounds for paramKey → Updates: parameter registry → Emits: ParameterUpdated(paramKey, oldValue, newValue, block.number)
- **Hardcoded Bounds (SAFETY PRINCIPLE):**

```
// These CANNOT be changed by any layer — immutable guardrails
uint256 constant MIN_QUORUM = 100;        // 1% minimum
uint256 constant MAX_QUORUM = 5000;       // 50% maximum
uint256 constant MIN_VOTING_DURATION = 7200;  // ~1 day @ 12s blocks
uint256 constant MAX_VOTING_DURATION = 604800; // ~1 week
uint256 constant MIN_CREDIT_CAP = 100;
uint256 constant MAX_CREDIT_CAP = 1000000;
```

- **Agent Prompt:** "Implement ParameterRegistry with immutable safety bounds. All parameter updates must pass through timelock validation. Include emergency freeze functionality."

Sub-Module 5.2: Layer 5 Integration Adapter

- **Functions:**
  - `receiveTimelockAdjustment(bytes32 paramKey, uint256 newValue)` → Callable ONLY by Layer 5 timelock contract address
  - `triggerEmergencyRevert()` → Resets ALL parameters to safe baseline (Layer 5 veto triggered) → Emits: EmergencyRevertTriggered(reasonHash)
- **Agent Prompt:** "Implement Layer5Adapter with strict address validation. Emergency revert must be atomic and immediate."

VERTICAL 6: EVENT & AUDIT MODULE (EAM)

Agent: EventAuditAgent

Responsibility: Comprehensive event emission, audit trail generation

Sub-Module 6.1: Event Schema

```
event ProposalCreated(bytes32 indexed proposalId, address indexed proposer, uint256 startBlock, uint256
↪ endBlock);
event VoteCast(bytes32 indexed proposalId, address indexed voter, uint256 totalCredits, int256[]
↪ votePowers);
event ProposalFinalized(bytes32 indexed proposalId, ProposalStatus status, uint256 totalParticipants);
event ProposalExecuted(bytes32 indexed proposalId, bool success, bytes returnData);
event CreditsLocked(address indexed voter, uint256 amount, bytes32 indexed proposalId);
event CreditsUnlocked(address indexed voter, uint256 amount, bytes32 indexed proposalId);
event ParameterUpdated(bytes32 indexed paramKey, uint256 oldValue, uint256 newValue, uint256
↪ blockNumber);
event EmergencyRevertTriggered(bytes32 reasonHash, uint256 timestamp);
event SybilDetected(address indexed identity, bytes32 evidenceHash);
```

- **Agent Prompt:** "Create a comprehensive event library with indexed parameters for efficient filtering. Include event aggregation helpers for Layer 4 oracle consumption."

Sub-Module 6.2: Audit Trail Generator

- **Functions:**
  - `generateStateMerkleRoot() → bytes32` → Periodic snapshot of all critical state for Layer 4 verification
  - `verifyStateTransition(bytes32 beforeRoot, bytes32 afterRoot, bytes proof) → bool` → For Layer 4 optimistic verification
- **Agent Prompt:** "Implement AuditTrail with merkle root generation. Optimize for gas efficiency while maintaining cryptographic integrity."

CROSS-VERTICAL INTERFACES & DEPENDENCIES

Interface Contracts (for agent-to-agent communication):

```
interface IVoiceCreditRegistry {
    function balanceOf(address identity) external view returns (uint256);
    function lockedOf(address identity) external view returns (uint256);
    function lockCredits(address identity, uint256 amount) external returns (bool);
    function unlockCredits(address identity, uint256 amount) external returns (bool);
}

interface IProposalRegistry {
```

```
    function getProposal(bytes32 id) external view returns (Proposal memory);
    function getStatus(bytes32 id) external view returns (ProposalStatus);
    function updateTally(bytes32 id, uint256 optionIndex, int256 delta) external;
    function storeVoteReceipt(bytes32 id, address voter, VoteReceipt calldata receipt) external;
}

interface IParameterRegistry {
    function getEffectiveQuorum() external view returns (uint256);
    function getVotingDuration() external view returns (uint256);
    function getCreditCap() external view returns (uint256);
    function applyUpdate(bytes32 key, uint256 value) external;
}

interface ILayer5Timelock {
    function scheduleParameterUpdate(bytes32 key, uint256 value, uint256 delay) external;
    function executeParameterUpdate(bytes32 key, uint256 value) external;
    function vetoPendingUpdate(bytes32 key) external;
}
```

AGENTIC CODING WORKFLOW

Recommended Agent Assignment:

Agent Name	Vertical(s)	Priority	Dependencies
CreditLedgerAgent	VCM	P0	None (foundation)
CreditMathAgent	VCM	P0	None (pure library)
ProposalLifecycleAgent	PLM	P0	VCM (for stake checks)
ProposalValidatorAgent	PLM	P1	PLM
VoteExecutionAgent	VEE	P0	VCM, PLM
TallyEngineAgent	VEE	P1	VEE
TreasuryExecutionAgent	TEM	P1	PLM, VEE
TreasuryGuardAgent	TEM	P2	TEM
ParameterInterfaceAgent	PIM	P1	All above
Layer5AdapterAgent	PIM	P1	PIM
EventAuditAgent	EAM	P2	All above

Implementation Order (Bottom-Up):

1. **Phase 1:** VCM (CreditMath → CreditRegistry) — Foundation
2. **Phase 2:** PLM (ProposalRegistry → ProposalValidator) — Core logic
3. **Phase 3:** VEE (VoteProcessor → TallyEngine) — Vote mechanics
4. **Phase 4:** TEM (ExecutionDispatcher → TreasuryGuard) — Execution safety
5. **Phase 5:** PIM (ParameterRegistry → Layer5Adapter) — Governance tuning
6. **Phase 6:** EAM (Events → AuditTrail) — Observability
7. **Phase 7:** Integration & End-to-End Testing

Testing Strategy per Agent:

- **Unit Tests:** Every function with 100% branch coverage
- **Invariant Tests:** Fuzzing against mathematical invariants
- **Integration Tests:** Cross-vertical atomic operation verification
- **Fork Tests:** Mainnet simulation for gas optimization